

1.8 PUZZLE PROBLEM

Algorithm

1. Read the initial state and goal state.
2. Place the initial state in the OPEN list.
3. Repeat until OPEN is empty:
 - Remove the best state.
 - If it is the goal, stop.
 - Generate all possible moves (up, down, left, right).
 - Ignore invalid or visited states.
 - Add new states to OPEN.
4. Display the solution path.

Pseudo Code

START

Read initial state

Read goal state

OPEN $\leftarrow$ initial state

CLOSED $\leftarrow$ empty

WHILE OPEN is not empty

 Remove first state from OPEN

 IF state = goal

 Print solution

 STOP

ENDIF

Generate child states

FOR each child

IF child not in CLOSED

Add child to OPEN

ENDIF

ENDFOR

Add current state to CLOSED

ENDWHILE

STOP

2.8 Queens Problem

Algorithm

1. Start from row 0.
2. Place a queen in a safe column.
3. Move to the next row.
4. If no safe position exists, backtrack.
5. Repeat until all queens are placed.

Pseudo code:

START

Function Solve(row)

IF row = 8

Print board

RETURN True

```
FOR each column
  IF Safe(row,column)
    Place Queen
    Solve(row+1)
    Remove Queen
  ENDIF
ENDFOR
END
```

Solve(0)

STOP

3. Water -Jug Problem

Algorithm

1. Start with both jugs empty.
2. Fill, empty, or pour water.
3. Generate all possible states.
4. Continue until the target amount is obtained.

Pseudo Code

START

Initial state $\leftarrow (0,0)$

WHILE goal not reached

Generate all valid moves

Visit unvisited states

ENDWHILE

Print solution

STOP

4.Crypt-Arithmetic

4. Crypt-Arithmetic Problem

Algorithm

1. Read the equation.
2. Identify unique letters.
3. Generate all digit permutations.
4. Replace letters with digits.
5. Check arithmetic correctness.
6. Print valid solution.

Pseudocode

START

Read equation

Extract unique letters

FOR every digit permutation

Assign digits

IF arithmetic is correct

Print solution

ENDIF

ENDFOR

STOP

5. Missionaries and Cannibals Problem

Algorithm

1. Start with all missionaries and cannibals on the left.
2. Generate valid boat moves.
3. Reject unsafe states.
4. Continue until everyone reaches the right side.

Pseudocode

START

Initial state $\leftarrow (3,3,\text{left})$

WHILE goal not reached

 Generate possible moves

 Check safety

 Add valid states

ENDWHILE

Print path

STOP

6. Vacuum Cleaner Problem

Algorithm

1. Start from current room.
2. If room is dirty, clean it.
3. Move to the next room.
4. Repeat until all rooms are clean.

Pseudocode

START

FOR every room

IF room is dirty

Clean room

ENDIF

Move next room

ENDFOR

STOP

7. Breadth First Search (BFS)

Algorithm

1. Insert source into queue.
2. Mark it visited.
3. Remove front node.

4. Visit all unvisited neighbours.
5. Repeat until queue is empty.

Pseudocode

START

Queue $\leftarrow$ source

WHILE Queue not empty

 node $\leftarrow$ dequeue

 Print node

 FOR every neighbour

 IF not visited

 Mark visited

 Enqueue

 ENDIF

 ENDFOR

ENDWHILE

STOP

8. Depth First Search (DFS)

Algorithm

1. Visit source.
2. Mark visited.

3. Visit each unvisited neighbour recursively.

Pseudocode

START

DFS(node)

 Mark visited

 Print node

 FOR every neighbour

 IF not visited

 DFS(neighbour)

 ENDIF

 ENDFOR

DFS(source)

STOP

9. Travelling Salesman Problem (TSP)

Algorithm

1. Start from source city.
2. Visit every city once.
3. Calculate total cost.
4. Select the minimum cost path.

Pseudocode

START

Minimum $\leftarrow$ infinity

FOR every permutation

 Calculate tour cost

 IF cost < minimum

 minimum $\leftarrow$ cost

 ENDIF

ENDFOR

Print minimum path

STOP

10. A* Algorithm

Algorithm

1. Put start node into OPEN list.
2. Compute
 - $g(n)$
 - $h(n)$
 - $f(n)=g+h$
3. Select node with minimum f .
4. Expand neighbours.
5. Repeat until goal.

Pseudocode

START

OPEN $\leftarrow$ start

WHILE OPEN not empty

 current $\leftarrow$ node with lowest f

 IF current = goal

 Print path

 STOP

 ENDIF

 Expand neighbours

 Update costs

ENDWHILE

STOP

11. Map Coloring (CSP)

Algorithm

1. Select an uncolored region.
2. Assign a color.
3. Check neighbouring regions.
4. If conflict occurs, backtrack.

5. Continue until all regions are colored.

Pseudocode

START

Color(region)

IF all regions colored

 Print solution

 RETURN

FOR each color

 IF Safe

 Assign color

 Color(next region)

 Remove color

 ENDIF

ENDFOR

STOP

12. Tic Tac Toe

Algorithm

1. Create board.
2. Alternate turns.
3. Check winner after every move.

4. Stop if winner or draw.

Pseudocode

START

Initialize board

WHILE game not over

 Player move

 Display board

 IF winner

 Print winner

 STOP

 ENDIF

 Change player

ENDWHILE

STOP

13. Minimax Algorithm

Algorithm

1. Generate all moves.
2. If terminal state, return score.
3. Maximizer selects maximum score.

4. Minimizer selects minimum score.

5. Return best move.

Pseudocode

Minimax(state,depth,max)

IF terminal

 RETURN score

IF maximizer

 RETURN maximum of children

ELSE

 RETURN minimum of children

ENDIF

14. Alpha-Beta Pruning

Algorithm

1. Perform Minimax search.
2. Maintain Alpha and Beta values.
3. Prune unnecessary branches.
4. Return best score.

Pseudocode

AlphaBeta(node,depth,alpha,beta,max)

IF terminal

 RETURN value

IF maximizer

FOR child

alpha=max(alpha,AlphaBeta)

IF alpha>=beta

BREAK

ELSE

FOR child

beta=min(beta,AlphaBeta)

IF beta<=alpha

BREAK

RETURN value

15. Decision Tree

Algorithm

1. Read dataset.
2. Calculate entropy.
3. Compute information gain.
4. Select best attribute.
5. Split dataset.
6. Repeat until leaf node.

Pseudocode

START

Read dataset

IF all same class

 RETURN leaf

Find attribute with highest gain

Split dataset

Recursively build tree

STOP

16. Feed Forward Neural Network

Algorithm

1. Initialize weights.
2. Read input.
3. Perform forward propagation.
4. Compute output.
5. Calculate error.
6. Update weights.
7. Repeat until convergence.

Pseudocode

START

Initialize weights

FOR each epoch

Input data

Forward propagation

Compute output

Calculate error

Update weights

ENDFOR

Print output

STOP